

Guide equipe - Module 1 : Fondations du framework Velt

1. Objectif du document

Ce document explique comment l'equipe va construire le Module 1 du framework Velt de maniere propre, academique et professionnelle.

Le Module 1 correspond aux fondations du framework. Il ne doit pas tout faire. Il doit construire les bases qui permettront ensuite de creer un vrai framework PHP moderne, modulaire, testable et evolutif.

Precision importante : le Module 1 seul ne livre pas encore toute l'experience finale **velt new mon-projet**. Cette experience complete arrive avec le Module 2, qui contient le skeleton et la developper experience. Le Module 1 livre les composants fondamentaux ; le Module 1 + le Module 2 livrent le MVP utilisable comme framework minimal.

L'objectif est que chaque membre de l'equipe comprenne :

- le role general du Module 1 ;
- le role exact de chaque sous-module ;
- comment les composants vont se connecter ;
- ce que chaque equipe doit livrer ;
- les prerequis techniques ;
- les regles de qualite attendues pour un projet academique.

2. Vision generale de Velt

Velt est un framework PHP moderne inspire par Laravel, mais avec une orientation plus modulaire et cross-platform.

L'idee principale est de construire Velt comme un ensemble de composants reutilisables. Laravel fonctionne deja beaucoup comme cela : routing, database, console, container, validation, etc. Velt va suivre cette logique, mais avec une identite propre.

Dans le MVP, Velt doit permettre de :

- initialiser un projet ;
- déclarer des routes ;
- créer des pages avec une syntaxe PHP déclarative ;
- rendre ces pages en HTML pour le Web ;
- rendre ces mêmes pages en JSON pour une preview mobile ;
- utiliser une CLI de type Artisan ;
- avoir une base database simple avec PDO ;
- préparer une architecture modulaire pour les prochains modules.

Le Module 1 est donc le socle technique. Si ce module est propre, tous les autres modules seront plus faciles à construire.

Le Module 1 est considéré réussi seulement si les composants peuvent fonctionner séparément et s'intégrer ensemble. Il ne suffit pas d'avoir des classes isolées : il faut prouver les flux essentiels comme route -> controller -> page UI -> HTML et preview session -> page UI -> JSON.

3. Principe fondamental : composants indépendants mais connectables

Chaque sous-module doit être pensé comme un composant autonome.

Cela veut dire :

- il a son propre repo GitHub ;
- il a son propre README ;
- il a ses propres tests ;
- il expose une API claire ;
- il dépend uniquement des composants nécessaires ;
- il ne doit pas connaître les détails internes des autres sous-modules.

Mais autonome ne veut pas dire isole. Les composants doivent pouvoir se connecter entre eux grace a des contrats et conventions communes.

Exemple :

- veltpHP/http ne doit pas connaitre toute l'implementation de veltpHP/ui.
- Mais un controller HTTP doit pouvoir retourner une page venant de veltpHP/ui.
- Donc on passe par des contrats simples comme RenderableInterface, JsonableInterface ou une Response.

Cette logique permet d'eviter un framework monolithique impossible a maintenir.

4. Les six sous-modules du Module 1

Le Module 1 est divise en six sous-modules. Chaque sous-module correspond a un futur repo GitHub.

Sous-module	Repo GitHub	Role
Sous-module 1	veltpHP/kernel	Noyau, contrats, container, configuration minimale
Sous-module 2	veltpHP/http	Request, Response, Router, Middleware
Sous-module 3	veltpHP/cli	CLI php velT, commandes et generateurs
Sous-module 4	veltpHP/ui	Composants UI declaratifs, rendu HTML et JSON
Sous-module 5	veltpHP/database	Connexion PDO, requetes preparees, model simple
Sous-module 6	veltpHP/preview	Sessions de preview, endpoints JSON, URL QR-ready

Ces six composants forment le coeur technique du MVP. Il ne faut pas ajouter de septieme sous-module dans le Module 1. Les sujets comme le skeleton applicatif, l'authentification, la validation, l'ORM complet, les migrations avancees, le cache ou les plugins appartiennent aux modules suivants.

5. Comment les composants vont se connecter

La connexion entre les composants doit suivre un ordre simple.

```
veltphp/kernel
|
|-- veltphp/http
|-- veltphp/ui
|-- veltphp/database
|-- veltphp/cli
    |
    |-- peut generer du code pour http, ui, database
|
|-- veltphp/preview
    |
    |-- utilise http + ui
```

Le composant le plus central est `veltphp/kernel`. Tous les autres composants peuvent l'utiliser, mais le kernel ne doit dependre d'aucun autre composant.

Chaque connexion entre composants doit passer par une API publique ou un contrat. Une equipe ne doit pas utiliser les classes internes d'un autre repo sans accord, sinon les composants deviennent impossibles a faire evoluer.

Exemple de flux Web

1. Le navigateur appelle `/login`.
2. `veltphp/http` capture la requete.
3. Le routeur trouve le controller.
4. Le controller retourne une page `Page::make(...)` venant de `veltphp/ui`.
5. Le renderer Web transforme la page en HTML.
6. `veltphp/http` renvoie une `Response HTML`.

Exemple de flux preview mobile

1. Le developpeur lance une commande preview.
2. `veltphp/preview` cree une session de preview.
3. Une URL est generee, par exemple `/api/preview/fgh123`.
4. Le mobile ou navigateur appelle cette URL.
5. `veltphp/http` recoit la requete.
6. `veltphp/preview` retrouve la page associee a la session.
7. `veltphp/ui` transforme la page en JSON.
8. `veltphp/http` renvoie une `JsonResponse`.

Exemple de flux CLI

1. Le developpeur tape `php velt make:feature auth`.
2. `veltphp/cli` lit les templates.
3. La CLI cree un controller, service, model, page UI et test.
4. Les fichiers generes respectent les conventions de Velt.

6. Syntaxe officielle des pages Velt

Le Module 1 doit respecter la syntaxe officielle Velt.

Une page Velt n'est pas un fichier HTML classique. Ce n'est pas non plus une syntaxe de tags inventee. C'est une declaration PHP sous forme d'objets.

Exemple :

```
<?php

declare(strict_types=1);

use Velt\UI\Components\Button;
use Velt\UI\Components\Card;
use Velt\UI\Components\Form;
use Velt\UI\Components\Input;
use Velt\UI\Components\Text;
use Velt\UI\Page;

return Page::make('Connexion')
    ->layout('auth')
    ->meta([
        'title' => 'Connexion - Velt App',
        'description' => 'Connectez-vous a votre compte',
    ])
    ->add(
        Card::make()
            ->class('w-full max-w-md mx-auto p-8 space-y-6')
            ->children([
                Text::make('Connexion')->as('h1'),
                Form::make()
                    ->method('POST')
                    ->action('/login')
                    ->csrf()
                    ->children([
                        Input::make('email')
                            ->label('Adresse email')
                            ->type('email')
                            ->required(),
```

```
        Button::make('Se connecter')
            ->type('submit')
            ->variant('primary'),
    ),
    ])
);
```

Cette page doit pouvoir produire :

- du HTML pour le Web ;
- du JSON pour la preview mobile.

C'est l'un des points les plus importants du projet.

7. Sous-module 1 : Kernel Contracts

Repo

veltphp/kernel

Role

Le kernel est le noyau du framework. Il definit les bases communes utilisees par tous les autres composants.

Il doit contenir :

- les contrats principaux ;
- le container minimal ;
- la configuration minimale ;
- les exceptions communes ;
- le bootstrap de l'application.

Prerequis techniques

Le membre responsable doit connaitre :

- PHP 8.2 ;
- namespaces PHP ;
- Composer ;
- autoload PSR-4 ;
- interfaces ;
- exceptions ;
- tests unitaires avec PHPUnit.

Livrables attendus

- `ApplicationInterface`
- `ContainerInterface`
- `ConfigRepositoryInterface`
- `RenderableInterface`
- `ArrayableInterface`
- `JsonableInterface`
- `Application`
- `ConfigRepository`
- container minimal ;
- bootstrap minimal ;
- README technique ;
- tests unitaires.

Issues du sous-module

- Issue 01 : initialiser le package Kernel.
- Issue 02 : creer les contrats fondamentaux.
- Issue 03 : implementer le container minimal.

- Issue 04 : ajouter configuration et bootstrap application.

Regle importante

Le kernel ne doit pas dependre de HTTP, UI, CLI, Database ou Preview. C'est la base. Si le kernel depend d'un autre module, l'architecture devient mauvaise.

Avant de considerer ce sous-module fini, il faut pouvoir instancier une application, lire une configuration avec notation par points, acceder au container et prouver par test que le kernel ne depend d'aucun autre composant Velt.

8. Sous-module 2 : HTTP Routing

Repo

veltphp/http

Role

Ce sous-module gere la couche HTTP du framework.

Il doit permettre de :

- capturer une requete ;
- creer une response ;
- declarer des routes ;
- executer un controller ;
- gerer des routes dynamiques ;
- retourner du HTML ou du JSON.

Prerequis techniques

Le membre responsable doit connaitre :

- fonctionnement HTTP ;
- methodes GET et POST ;

- status codes ;
- headers ;
- superglobales PHP ;
- controllers ;
- middleware ;
- tests d'integration simples.

Livrables attendus

- Request
- Response
- JsonResponse
- Router
- support /api/preview/{id}
- middleware minimal ;
- normalisation des retours controller ;
- gestion propre des erreurs HTTP ;
- tests pour 200, 404 et 405.

Issues du sous-module

- Issue 01 : creer Request et Response.
- Issue 02 : implementer le routeur MVP.
- Issue 03 : ajouter middleware et dispatch controller.
- Issue 04 : normaliser les retours controller et erreurs HTTP.

Connexion avec les autres modules

HTTP depend de veltphp/kernel.

Il pourra recevoir une page venant de veltphp/ui, mais il ne doit pas connaître l'intérieur du moteur UI. Il doit seulement savoir transformer le résultat en réponse.

Avant de considérer ce sous-module fini, il faut vérifier qu'un contrôleur peut retourner une Response, une string HTML, un tableau JSON ou un objet renderable, et que chaque retour devient une réponse HTTP correcte.

9. Sous-module 3 : CLI Generators

Repo

veltphp/cli

Role

La CLI est l'outil de productivité du framework. Elle doit donner une expérience proche de Laravel Artisan.

Commandes MVP :

- php velt list
- php velt help
- php velt make:feature auth
- php velt make:controller
- php velt make:model
- php velt make:test
- php velt serve
- php velt preview

Prerequis techniques

Le membre responsable doit connaître :

- scripts CLI en PHP ;

- arguments et options ;
- generation de fichiers ;
- templates ;
- manipulation de chemins ;
- Composer bin ;
- tests sur filesystem temporaire.

Livrables attendus

- executable bin/velt ;
- systeme de commandes ;
- generateur de feature ;
- generateur de test ;
- commande serve ;
- distribution Composer via vendor/bin/velt ;
- documentation des commandes.

Issues du sous-module

- Issue 01 : initialiser la CLI php velt.
- Issue 02 : creer make:feature.
- Issue 03 : creer serve et make:test.
- Issue 04 : preparer distribution Composer de la CLI.

Connexion avec les autres modules

La CLI genere des fichiers compatibles avec :

- veltphp/http pour controllers/routes ;
- veltphp/ui pour pages ;

- `veltphp/database` pour models ;
- `veltphp/preview` pour preview.

Elle ne doit pas copier la logique interne des autres modules. Elle doit seulement generer du code qui respecte leurs APIs publiques.

Avant de considerer ce sous-module fini, il faut verifier que `php bin/velt list` fonctionne, que `vendor/bin/velt list` fonctionne apres installation Composer locale, que les commandes retournent des codes de sortie corrects et que les generateurs n'ecrasent pas des fichiers sans option explicite.

10. Sous-module 4 : UI Rendering

Repo

`veltphp/ui`

Role

Ce module est une partie tres importante de l'identite de Velt. Il permet de creer une interface avec du PHP declaratif.

Il doit fournir :

- `Page`
- `Card`
- `Text`
- `Alert`
- `Form`
- `Input`
- `Button`
- `Link`
- `WebRenderer`
- `JsonRenderer`

- ViewFactory
- ViewNotFoundException

Prerequis techniques

Le membre responsable doit connaitre :

- programmation orientee objet PHP ;
- method chaining ;
- tableaux PHP ;
- serialization JSON ;
- escaping HTML ;
- notions de composants UI ;
- tests snapshot simples.

Livrables attendus

- composants chainables ;
- representation interne en arbre ;
- toArray() ;
- rendu HTML ;
- rendu JSON ;
- chargement de fichier .velt.php ;
- support view('auth.login') ou equivalent ;
- tests sur tous les composants.

Issues du sous-module

- Issue 01 : creer Page et composants UI.
- Issue 02 : implementer WebRenderer HTML.

- Issue 03 : implementer JsonRenderer preview.
- Issue 04 : ajouter ViewFactory et chargement des pages Velt.

Connexion avec les autres modules

UI depend de veltphp/kernel.

Il est utilise par :

- veltphp/http pour rendre une page Web ;
- veltphp/preview pour produire le JSON mobile ;
- veltphp/cli pour generer des pages.

UI ne doit pas dependre de database ou auth dans le Module 1.

Avant de considerer ce sous-module fini, il faut verifier qu'une page Velt peut etre declaree en PHP, chargee depuis un fichier, rendue en HTML, rendue en JSON, et que les textes/attributs HTML sont echappes correctement.

11. Sous-module 5 : Database PDO

Repo

veltphp/database

Role

Ce sous-module fournit une couche database simple pour le MVP.

Attention : ce n'est pas encore l'ORM complet. L'ORM avance arrive dans le Module 3.

Dans le Module 1, on veut seulement :

- connecter PDO ;
- executer des requetes preparees ;
- avoir un helper DB ;
- avoir un model simple.

Prerequis techniques

Le membre responsable doit connaitre :

- PDO ;
- SQL ;
- requetes preparees ;
- transactions ;
- configuration database ;
- SQLite pour les tests ;
- securite basique SQL injection.

Livrables attendus

- DatabaseManager
- ConnectionFactory
- helper DB
- select, first, statement, transaction
- Model simple ;
- integration avec ConfigRepositoryInterface ;
- tests SQLite.

Issues du sous-module

- Issue 01 : creer Database Manager PDO.
- Issue 02 : ajouter Query Helper securise.
- Issue 03 : creer BaseModel MVP.
- Issue 04 : integrer database avec configuration kernel.

Connexion avec les autres modules

Database depend de veltphp/kernel.

Il peut etre utilise par les features generees par la CLI. Mais le moteur UI et le routeur ne doivent pas dependre directement de database.

Avant de considerer ce sous-module fini, il faut verifier qu'une connexion SQLite peut etre creee depuis une configuration kernel, que les DSN MySQL/PostgreSQL peuvent etre construits correctement, que les requetes utilisent des placeholders, et que les erreurs de configuration sont explicites.

12. Sous-module 6 : Preview API

Repo

veltphp/preview

Role

Ce sous-module permet de previsualiser une page Velt en JSON.

Il sert a prouver l'idee cross-platform du framework : une meme page peut etre rendue pour le Web et pour un client mobile.

Prerequis techniques

Le membre responsable doit connaitre :

- APIs JSON ;
- routing HTTP ;
- fichiers JSON ;
- generation d'identifiants ;
- architecture controller ;
- integration entre plusieurs packages.

Livrables attendus

- PreviewSession
- PreviewSessionStore
- endpoint GET /api/preview/{id}
- endpoint GET /api/session/{id}
- generation d'URL preview ;
- payload compatible QR code ;
- test de flux preview de bout en bout ;
- tests integration.

Issues du sous-module

- Issue 01 : creer PreviewSession Store.
- Issue 02 : ajouter endpoints preview.
- Issue 03 : integrer payload QR code CLI.
- Issue 04 : valider le flux preview de bout en bout.

Connexion avec les autres modules

Preview depend de :

- veltphp/kernel
- veltphp/http
- veltphp/ui

Il peut etre appele par veltphp/cli pour creer une session de preview.

Preview est un module d'integration. C'est normal qu'il depende de plusieurs composants.

Avant de considerer ce sous-module fini, il faut verifier le scenario complet : une session preview pointe vers une page Velt, une requete GET /api/preview/{id} charge cette page, le JSON contient screen, schemaVersion et components, et les erreurs 404 sont propres.

16. Ce qui n'est pas dans le Module 1

Pour garder le MVP realiste, le Module 1 ne doit pas inclure :

- ORM complet type Eloquent ;
- authentication complete ;
- validation avantee ;
- migrations avancees ;
- microservices ;
- NoSQL ;
- hot reload mobile ;
- WebSocket ;
- plugins ;
- deployment production complet.

Ces sujets sont importants, mais ils appartiennent aux modules suivants.